

SOCIAL NETWORKS

NPTEL Online Course • Prof. Sudarshan Iyengar, IIT Ropar

Week 7 — Question Paper (Comprehension-Based MCQ / MSQ)

Comprehension 1 — How a New App Spreads Through a Market

Case Study

An analyst studies a large wholesale market whose vendors are linked by a network of trading partnerships. A new digital-payment app is introduced, and it spreads from vendor to vendor: a few early adopters start using it, some of their partners follow, then some partners of those partners, and so on. The analyst notes that this is not a single chain but a branching spread one adopter can seed several others which is why the phenomenon is called diffusion rather than travel.

To model each vendor's choice, the analyst uses a simple coordination game with two actions: keep using cash, with payoff b per partner who also uses cash, or adopt the app, with payoff a per partner who also uses the app. If a fraction p of a vendor's partners have adopted the app, the vendor's payoff from adopting is $p \cdot a$ and from staying on cash is $(1-p) \cdot b$. A vendor switches to the app once $p \cdot a \geq (1-p) \cdot b$, which rearranges to a clean threshold on p . As more partners adopt, a vendor edges past this threshold an effect called social reinforcement.

Diffusion does not always sweep the whole market. Tight-knit groups can resist: the analyst defines a cluster to have density D if every vendor in it has at least a fraction D of its partners inside the cluster, and finds that a cluster of high enough density blocks the spread. To get a stalled product moving, one can either raise the app's payoff or seed a few well-connected key vendors rather than advertising to everyone. The same market also faces a separate question: the vendors privately dislike a new market fee and would join a protest, but each vendor has its own intrinsic threshold for joining and because each vendor knows only its own partners' thresholds, a protest that could in principle succeed may never get off the ground.

Questions 1–10

Q1. [MCQ] Why does the analyst describe the app's spread as “diffusion” rather than “travel”?

- A. Because the app moves along one single chain of vendors from start to finish.
- B. Because the spread is controlled centrally and pushed out to all vendors at once.
- C. Because adoption branches outward — each adopter can seed several others in turn.
- D. Because only vendors with no trading partners can ever adopt the new app.

Answer: C

Reference: Lecture 89 – Diffusion in Networks (≈01:01–02:07)

Solution: Diffusion is a branching process: an adopter can pass the behaviour to several partners, who each pass it on further, so it fans out through the network rather than following one path. It is not a single chain, not a central broadcast, and isolated vendors are not the point.

Q2. [MCQ] A vendor switches to the app when $p \cdot a \geq (1-p) \cdot b$, where p is the fraction of partners already on the app. What threshold on p does this give?

- A. $p \geq b / (a + b)$
- B. $p \geq a / (a + b)$
- C. $p \geq (a + b) / b$
- D. $p \geq a / b$

Answer: A

Reference: Lecture 91 – Modeling Diffusion (Continued) (≈00:46–01:58)

Solution: From $p \cdot a \geq (1-p) \cdot b$ expand to $p \cdot a \geq b - p \cdot b$, so $p \cdot (a + b) \geq b$, giving $p \geq b / (a + b)$. With app-payoff $a = 3$ and cash-payoff $b = 2$, that threshold is $2/5 = 0.4$.

Q3. [MCQ] The app's payoff is $a = 3$ per adopting partner and cash's is $b = 2$ per cash partner. A vendor has 20 partners: 6 on the app and 14 on cash. Which action gives the higher payoff?

- A. The app, by 18 to 14, because more partners use cash overall.
- B. A tie, 28 each, so the vendor is indifferent between the two.
- C. The app, by 28 to 18, because its per-partner payoff is larger.
- D. Cash, by 28 to 18, so the vendor keeps using cash.

Answer: D

Reference: Lecture 90 – Modeling Diffusion (≈05:12–06:15)

Solution: The app payoff is $6 \times 3 = 18$ and the cash payoff is $14 \times 2 = 28$. Since $28 > 18$, the vendor stays on cash. (Equivalently, $p = 6/20 = 0.3$ is below the 0.4 threshold.)

Q4. [MCQ] What does “social reinforcement” mean in this model?

- A. Each additional partner who adopts pushes a vendor closer to its switching threshold.
- B. A vendor's decision is fixed at the start and cannot change as partners adopt.
- C. Vendors deliberately do the opposite of whatever most of their partners do.
- D. The market authority reinforces adoption by ordering every vendor to switch.

Answer: A

Reference: Lecture 91 – Modeling Diffusion (Continued) (≈02:49–04:31)

Solution: Social reinforcement is the cumulative pull of peers: as more of a vendor's partners adopt, the fraction p rises and the vendor is nudged past its threshold. Decisions are not fixed, vendors are not contrarian by rule, and the effect comes from peers rather than an order from above.

Q5. [Match the following] Match each term in Column A with its description in Column B.

Column A (term)	Column B (description)
1. Diffusion	a. the new behaviour ends up adopted by every node
2. Complete cascade	b. a few well-connected seeds chosen to start the spread

3. Cluster of density D	c. a behaviour branching from node to node through the network
4. Key people	d. every node in it has at least a fraction D of its neighbours inside it

- A. 1-c, 2-a, 3-b, 4-d
- B. 1-a, 2-c, 3-d, 4-b
- C. 1-c, 2-a, 3-d, 4-b
- D. 1-c, 2-d, 3-a, 4-b

Answer: C (1-c, 2-a, 3-d, 4-b)

Reference: Lectures 89–93 – Diffusion, cascades, communities, and key people

Solution: Diffusion is the branching spread of a behaviour (1-c); a complete cascade is when every node adopts the new behaviour (2-a); a cluster has density D when each member keeps at least a fraction D of its neighbours inside the cluster (3-d); and key people are well-connected seeds used to launch the spread (4-b).

Q6. [MCQ] Under what condition does the theorem say a cascade with threshold q cannot complete?

- A. When there is no cluster anywhere whose density exceeds the value $1 - q$.
- B. When the network contains a cluster of density greater than $1 - q$.
- C. When every node in the network has a threshold q strictly below one half.
- D. When the seed set is larger than the densest cluster in the whole network.

Answer: B

Reference: Lecture 93 – Cascade and Clusters (≈06:34–10:27)

Solution: A cluster of density greater than $1 - q$ stops the cascade: each member then has more than $1 - q$ of its neighbours inside the cluster, so fewer than q of its neighbours can be outside adopters, and its threshold q is never met. The absence of such a cluster is exactly the opposite condition, and the threshold value or seed-set size alone is not the criterion.

Q7. [MCQ] The analyst suggests seeding a few “key people.” What is the idea behind this strategy?

- A. Convince a few highly connected vendors, whose adoption then pulls in many others.
- B. Convince the vendors with the fewest partners, since they are easiest to reach.
- C. Advertise equally to every vendor at once so that none is left uninformed.
- D. Wait for the densest cluster to adopt first and let it spread to the rest.

Answer: A

Reference: Lecture 92 – Impact of Communities on Diffusion (≈05:42–08:13)

Solution: Targeting well-connected vendors (viral marketing) means a few adoptions can ripple out to much of the network, like convincing the most influential member of a group. Seeding the least-connected vendors, blanket advertising, or waiting on the densest cluster (which actually resists) are all weaker strategies.

Q8. [MCQ] In the collective-action model, what is a vendor's "intrinsic threshold"?

- A. The payoff a vendor earns for each partner who joins the protest with it.
- B. The number of partners a vendor has, which caps how many it can recruit.
- C. The fraction of the market fee a vendor is personally willing to pay each month.
- D. The minimum number of vendors, including itself, that must join before it will.

Answer: D

Reference: Lecture 94 – Knowledge, Thresholds and Collective Action (≈04:30–end)

Solution: Each vendor has a personal cut-off: it joins the protest only once at least that many vendors (counting itself) are in. It is a count of participants, not a payoff, a partner count, or a sum of money.

Q9. [MCQ] Three vendors have protest thresholds 2, 3, and 4, but the one needing 4 has only 2 trading partners. Why does no protest happen?

- A. Its high payoff means the vendor needing 4 prefers to protest entirely on its own.
- B. The vendor needing 4 can never reach it, so the others, seeing this, also drop out.
- C. All three thresholds are met at once, so the protest succeeds rather than failing.
- D. The vendor needing only 2 has too many partners and is afraid to act first.

Answer: B

Reference: Lecture 94 – Knowledge, Thresholds and Collective Action (≈05:58–end)

Solution: With only 2 partners, the vendor needing 4 can never reach its threshold, so it will not join. The vendor needing 3 sees that and backs out, and then the vendor needing 2 follows — the willingness collapses even though a protest was in principle possible. The others do not all clear their thresholds, and it is the limited knowledge, not over-connection, that stalls things.

Q10. [MSQ] Which statements about diffusion and collective action are correct? (Select all that apply.)

- I. A vendor adopts the app once at least a fraction $b/(a+b)$ of its partners have.
 - II. A sufficiently dense cluster can stop a cascade from completing.
 - III. Incomplete knowledge of others' thresholds can prevent a feasible protest.
 - IV. Raising the app's payoff always makes a complete cascade impossible.
- A. I, II, IV
 - B. I, II, III
 - C. II, III, IV
 - D. I, IV

Answer: B (statements I, II, III)

Reference: Lectures 91–94 – Thresholds, clusters, key people, collective action

Solution: The switching threshold $b/(a+b)$ (I), the blocking effect of a dense cluster (II), and the way incomplete knowledge can stall a feasible protest (III) are all correct. Raising the payoff makes a complete cascade easier, not impossible, so IV is false.

Comprehension 2 — Simulating the Adoption Cascade in NetworkX

Case Study

A developer codes the adoption cascade to test four ideas from the chapter. The underlying network is a random Erdős–Rényi graph, and because the same graph must be reused across several experiments it is saved once and reloaded each time. Every node carries an attribute action set to 'B' (the old behaviour) initially; a chosen seed set is then flipped to 'A' (the new behaviour). In the drawing, 'A' is shown green and 'B' red.

Each round, every node re-weighs its options: it counts how many neighbours currently hold each action and compares payoffs, where the payoff of A is a times the number of A-neighbours and the payoff of B is b times the number of B-neighbours. Each node then takes whichever action scores higher, and the new decisions are applied to the whole graph at once. The loop ends when the cascade settles either every node is on A or every node is on B or when 100 rounds pass, a cap that guards against cases where nodes keep flipping forever. A cascade is called complete when every node has adopted A.

With this machine the developer demonstrates four big ideas: (1) raising the payoff a until a stalled cascade completes; (2) keeping the payoff fixed but changing which key nodes are seeded; (3) showing a cascade fill one community yet fail to cross into another; and (4) confirming that a cluster of density above $1-q$ blocks the cascade. Key fragments:

```
import networkx as nx
G = nx.erdos_renyi_graph(10, 0.5)
nx.write_gml(G, 'random_graph.gml')      # save once
G = nx.read_gml('random_graph.gml')     # reuse the same graph

set_all_B(G)                            # every node starts on action B
set_A(G, [3, 7])                        # seed nodes adopt action A

# one round: each node re-weighs its options
payoff_A = a * num_A                    # num_A = neighbours currently on A
payoff_B = b * num_B
dict1[each] = 'A' if payoff_A >= payoff_B else 'B'
reset_node_attributes(G, dict1)

# stop on a complete cascade, or after 100 rounds
if terminate_1('A', G) or terminate_1('B', G) or count >= 100:
    break
```

Questions 1–10

Q1. [MCQ] Why does the code save the Erdős–Rényi graph with `write_gml` and reload it with `read_gml`?

- A. Because `erdos_renyi_graph` can only be called once per Python session in total.
- B. So every experiment runs on the identical graph, making the comparisons fair.
- C. Because GML is the only file format that NetworkX is able to draw on screen.
- D. To convert the undirected graph into a directed one before the cascade starts.

Answer: B

Reference: Lecture 96 – The Base Code (≈00:05–02:01)

Solution: Saving once and reloading guarantees that the four idea-experiments all use the same random graph, so any difference in outcome is due to the change being tested, not to a fresh random graph. The generator can be called many times, GML is not tied to drawing, and reloading does not change the graph's direction.

Q2. [MCQ] How does the code set up the starting state of the cascade?

- A. It assigns every node action 'A', then flips the seed set to 'B' for contrast.
- B. It assigns each node a random action drawn independently from 'A' and 'B'.
- C. It sets every node to 'B' with `set_all_B`, then sets the seed nodes to 'A' with `set_A`.
- D. It deletes all nodes except the seeds, then regrows the graph around them.

Answer: C

Reference: Lecture 96 – The Base Code (≈02:01–05:23)

Solution: `set_all_B` makes every node hold the old action B, and `set_A` then flips the chosen seed list to the new action A. The actions are not assigned the other way round, not randomised, and no nodes are deleted.

Q3. [MCQ] In `recalculate_options`, when does a node adopt action A?

- A. When $a\text{-num_A} \geq b\text{-num_B}$, i.e. its A-payoff is at least its B-payoff.
- B. When $\text{num_A} \geq \text{num_B}$, regardless of the two payoff values a and b.
- C. When $a\text{-num_A}$ is strictly less than $b\text{-num_B}$ for that particular node.
- D. When the node has strictly more than five neighbours on action A.

Answer: A

Reference: Lecture 96 – The Base Code (≈10:55–15:29)

Solution: The node compares weighted payoffs: $\text{payoff_A} = a\text{-num_A}$ and $\text{payoff_B} = b\text{-num_B}$, adopting A when $\text{payoff_A} \geq \text{payoff_B}$. A bare count of neighbours ignores the payoffs a and b, the inequality direction in another option is reversed, and there is no fixed five-neighbour rule.

Q4. [MCQ] What does `num_A` represent for a given node in this code?

- A. The total number of neighbours that node has, on either action.
- B. The payoff value a associated with adopting the new action A.
- C. The round number, counting how many times the node has flipped so far.
- D. The count of that node's neighbours that currently hold action A.

Answer: D

Reference: Lecture 96 – The Base Code (≈12:58–15:29)

Solution: `find_neighbours` counts how many of the node's neighbours currently hold a given action, so `num_A` is the number of A-neighbours. It is not the node's total degree, not the payoff a, and not a round counter.

Q5. [MCQ] Why does the loop include the condition count ≥ 100 alongside the all-A and all-B checks?

- A. Because the graph is guaranteed to converge in well under one hundred rounds.
- B. To cap the run when nodes keep flipping and the cascade never converges.
- C. Because exactly one hundred nodes must adopt A before the run may stop.
- D. To redraw the network on screen once for each of the hundred rounds run.

Answer: B

Reference: Lecture 97 – Coding the First Big Idea (≈03:36–06:24)

Solution: Some configurations never settle — nodes flip back and forth — so the 100-round cap forces the loop to stop instead of running forever. Convergence is not guaranteed, the graph has 10 nodes (not 100 adopters), and the cap is about termination, not redrawing.

Q6. [MCQ] How does the code decide that a cascade is “complete”?

- A. It checks count == 100, treating any full-length run as a complete cascade.
- B. It checks whether the seed set grew by at least one node during the run.
- C. terminate_1('A', G) returns 1 only when every node in the graph holds action A.
- D. It compares the number of A-nodes with the number of B-nodes at the end.

Answer: C

Reference: Lecture 97 – Coding the First Big Idea (≈04:09–05:40)

Solution: A cascade is complete when the new action has taken over entirely, which terminate_1('A', G) detects by returning 1 exactly when all nodes hold A. Reaching 100 rounds, seed growth, or a simple A-vs-B count does not by itself mean a complete cascade.

Q7. [Arrange in order] Arrange the steps of the cascade simulation in the correct order.

Steps (in scrambled order):

- (i) Seed a few nodes with the new action A.
 - (ii) Build an Erdős–Rényi graph and set every node to the old action B.
 - (iii) Each round, recalculate every node's action from its neighbours' payoffs.
 - (iv) Stop when the cascade settles (all A or all B) or after 100 rounds.
 - (v) Report whether the cascade is complete (every node on A).
- A. ii → iii → i → iv → v
 - B. i → ii → iii → iv → v
 - C. ii → i → iv → iii → v
 - D. ii → i → iii → iv → v

Answer: D (ii → i → iii → iv → v)

Reference: Lectures 96–97 – Base code and the cascade loop

Solution: Build the graph and set everyone to B (ii), seed the new action A on a few nodes (i), then each round recalculate actions from neighbours' payoffs (iii), stop when the cascade settles or 100 rounds pass (iv), and finally report whether it was a complete cascade (v).

Q8. [MCQ] Starting from seeds 0 and 1, the cascade dies out at payoffs $a = 6, 7, 8, 9$ but completes at $a = 10$. Which “big idea” does this demonstrate?

- A. Increasing the new action's payoff can turn a stalled cascade into a complete one.
- B. Choosing different seed nodes is what flips the cascade from partial to complete.
- C. A dense cluster is blocking the cascade regardless of the payoff value chosen.
- D. Two separate communities each adopt a different action and never merge at all.

Answer: A

Reference: Lecture 97 – Coding the First Big Idea (≈00:34–02:14)

Solution: Here the seeds are fixed and only the payoff a is raised; once it reaches 10 the cascade completes, illustrating the first idea — increasing the payoff. Changing seeds, cluster blocking, and community splits are the other three ideas, not what is varied in this run.

Q9. [MCQ] In the community experiment, the developer keeps the payoff fixed and seeds one community. What is observed?

- A. The cascade jumps straight to the far community and skips its own entirely.
- B. The cascade dies out instantly, leaving even the seeded community on action B.
- C. Both communities adopt A simultaneously in the very first round of the run.
- D. The cascade fills the seeded community but fails to cross into the other one.

Answer: D

Reference: Lecture 99 – Coding the Third Big Idea: Communities (≈06:05–end)

Solution: The new action spreads through the seeded community but the sparse links to the other community are not enough to carry it across, so one community ends on A and the other stays on B. It does not skip its own community, die out at once, or convert both communities instantly.

Q10. [MSQ] Which statements about the simulation are correct? (Select all that apply.)

- I. The same saved graph is reused so the four experiments are comparable.
 - II. A node adopts A when $a \cdot \text{num_A}$ is at least $b \cdot \text{num_B}$.
 - III. The 100-round cap stops runs in which nodes never stop flipping.
 - IV. A cascade is complete whenever the run reaches exactly 100 rounds.
- A. I, II, IV
 - B. II, III, IV
 - C. I, II, III
 - D. I, IV

Answer: C (statements I, II, III)

Reference: Lectures 96–97 – Base code, payoff rule, and termination

Solution: Reusing the saved graph (I), the weighted-payoff adoption rule (II), and the 100-round cap for non-converging runs (III) are all correct. Reaching 100 rounds means the run was stopped, not that the cascade completed, so IV is false.

End of Week 7 Question Paper — 20 questions • 2 comprehensions